Error Amplification in Entity Resolution Pipelines: A Formal Analysis of Stage-Wise Quality Propagation

Anonymous Author(s)

Entity resolution (ER) pipelines typically consist of three sequential stages—blocking, matching, and clustering—yet each stage is almost always evaluated in isolation. This creates a critical blind spot: practitioners lack a principled way to determine which stage to improve for maximum end-to-end quality gain. We present PipeER, a framework for modeling error propagation across ER pipeline stages. PipeER proves that end-to-end recall is bounded by blocking recall (an irrecoverable error), fits a parametric Error Propagation Model (EPM) that predicts end-to-end F1 from per-stage metrics, and defines Error Amplification Factors (EAFs) for identifying pipeline bottlenecks. A Stage-Optimal Allocation (SOA) algorithm directs improvement effort based on the EPM. Experiments on six standard benchmarks with 648 pipeline configurations show that the EPM achieves $R^2 = 0.975$ on held-out data, the hard recall bound holds in 99.5% of configurations, and SOA allocation yields 25–88% higher improvement efficiency compared to uniform allocation. Critically, per-method bottleneck analysis reveals that the bottleneck stage depends on the matching method used: blocking dominates for strong matchers (random forest), while clustering dominates for weaker ones (rule-based, SBERT), highlighting that pipeline optimization recommendations must be method-specific.

1 Introduction

Entity resolution (ER)—the task of identifying records across data sources that refer to the same real-world entity—is a foundational problem in data integration [5, 3]. Modern ER systems are typically implemented as multi-stage pipelines consisting of *blocking* (filtering candidate pairs to reduce quadratic comparison cost), *matching* (classifying candidate pairs as match or non-match), and *clustering* (grouping matched records into entity clusters) [4].

Despite decades of research advancing individual stages—from classical blocking techniques [11] to deep learning matchers [10, 9]—a fundamental gap persists: **ER pipeline stages are almost always evaluated in isolation**. Blocking is evaluated by pairs completeness, matching by F1 score, and clustering by cluster-level metrics, with little understanding of how per-stage quality translates to end-to-end performance. This creates practical problems: when a pipeline produces poor results, there is no principled way to determine whether to invest in better blocking, better matching, or better clustering.

The core insight motivating this work is that errors at different pipeline stages have **fundamentally asymmetric propagation behavior**. Blocking errors (missed true pairs) are *irrecoverable*—a true match excluded from the candidate set can never be recovered by downstream stages. Matching false negatives can be partially compensated by transitive closure during clustering (if A matches B and B matches C, the cluster $\{A, B, C\}$ forms even if A–C was missed). Meanwhile, matching false positives can be *amplified* by clustering when connected components merge distinct entities.

Our contributions are:

- We prove that end-to-end recall is bounded by blocking recall ($R_{\text{e2e}} \leq \text{PC}$), establishing blocking as an irrecoverable error source, and validate this bound empirically across 648 pipeline configurations on six benchmark datasets.
- We fit a parametric Error Propagation Model (EPM) that predicts end-to-end F1 from per-stage metrics with $R^2 = 0.975$, and we define Error Amplification Factors (EAFs) for empirical bottleneck identification.
- We propose a Stage-Optimal Allocation (SOA) algorithm that achieves 25–88% higher improvement efficiency compared to uniform allocation across all six datasets.

- We conduct a comprehensive per-method bottleneck analysis revealing that the bottleneck stage is method-dependent—blocking for strong matchers, clustering for weak ones—providing actionable guidance for practitioners.
- We provide the most systematic empirical study of ER pipeline stage interactions to date, spanning 3 blocking methods, 4 matching methods, 3 clustering methods, and 6 benchmark datasets.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the PipeER framework. Section 4 describes the experimental setup and results. Section 5 discusses findings and limitations. Section 6 concludes.

2 Related Work

End-to-end ER surveys. Christophides et al. [4] provide a comprehensive survey of ER for big data, covering all pipeline stages independently. Binette and Steorts [2] present a unified statistical treatment of ER, emphasizing the importance of propagating uncertainty through all stages. However, neither work formally models how per-stage errors compound into end-to-end quality or provides optimization frameworks for stage-wise improvement allocation. Our Error Propagation Model fills this gap.

Blocking evaluation. Papadakis et al. [11] survey blocking and filtering techniques, establishing standard metrics such as pairs completeness (PC) and reduction ratio (RR). Their evaluation framework treats blocking in isolation from downstream stages. Our work extends this by formally quantifying blocking's *downstream impact*, proving that blocking recall is a hard upper bound on end-to-end recall.

Clustering evaluation. Hassanzadeh et al. [7] present a framework for evaluating clustering algorithms in duplicate detection, defining cluster-level precision and recall metrics. Our work builds on their metrics but places clustering within the full pipeline context, formally analyzing how matching quality—especially false-positive topology—determines clustering behavior.

ER benchmark difficulty. Papadakis et al. [12] critically re-evaluate ER benchmark datasets using linearity and complexity measures, showing many benchmarks are too easy for deep learning methods. Our analysis complements theirs by revealing how the bottleneck stage varies across datasets of different difficulty levels and across matching methods of different quality.

Pipeline optimization for ML. Siddiqi et al. [14] present SAGA, a framework for optimizing data cleaning pipelines via AutoML-style search. SAGA treats the pipeline as a black box. Our approach differs fundamentally: we provide a *white-box* parametric model of error propagation specific to ER pipelines, enabling bottleneck identification without expensive search.

Deep learning for ER. Mudgal et al. [10] (DeepMatcher) and Li et al. [9] (Ditto) demonstrate that deep learning substantially improves matching quality. Our framework is method-agnostic and analyzes how improvements at the matching stage translate to end-to-end gains, providing guidance on whether investing in better matching models is worthwhile relative to improving blocking or clustering.

Progressive ER. Galhotra et al. [6] propose progressive blocking that uses partial ER output to refine blocking in a feedback loop. This recognizes the interaction between blocking and matching but focuses on a specific algorithmic solution rather than providing a general framework for understanding stage interactions.

3 Method

We present PipeER, a framework with three components: (1) a formal hard recall bound (Theorem 1), (2) a parametric Error Propagation Model (EPM) fit to pipeline data, and (3) a Stage-Optimal Allocation (SOA) algorithm for directing improvement effort.

3.1 Problem Setup

Consider an ER pipeline operating on two record sets A and B. Let $P = \{(r_i, r_j) \mid r_i \in A, r_j \in B\}$ denote all record pairs and $M \subseteq P$ denote the set of true match pairs.

Stage 1: Blocking. Blocking produces a candidate set $C \subseteq P$. We measure:

$$\text{PC} = \frac{|C \cap M|}{|M|}, \qquad \text{RR} = 1 - \frac{|C|}{|P|}. \tag{1}$$

Stage 2: Matching. Given candidates C, matching classifies each pair, producing predicted matches $D \subseteq C$. We measure:

$$\text{MP} = \frac{|D \cap M|}{|D|}, \qquad \text{MR} = \frac{|D \cap M|}{|C \cap M|}. \tag{2}$$

Stage 3: Clustering. Given predicted matches D, clustering groups records into entity clusters $\mathcal{E} = \{E_1, \ldots, E_k\}$. We measure cluster precision (CP: fraction of within-cluster pairs that are true matches) and cluster recall (CR: fraction of true match pairs co-clustered).

3.2 Hard Recall Bound

The key theoretical result is that end-to-end recall is bounded by blocking recall:

Theorem 1 (Hard Recall Bound)**.** *End-to-end recall satisfies* $R_{\text{e2e}} \leq \text{PC}$.

Proof. A true match pair $(r_i, r_j) \in M$ can only contribute to end-to-end recall if it is co-clustered. Co-clustering requires the pair to be connected (directly or transitively) in the predicted match graph D. Since $D \subseteq C$, only pairs in the candidate set C can appear in D. Therefore, any true match pair not in C (i.e., $(r_i, r_j) \in M \backslash C$) cannot be recovered by any downstream matching or clustering, giving $R_{\text{e2e}} \leq |C \cap M|/|M| = \text{PC}$. □

This bound is tight: it is achieved when matching and clustering are perfect on the candidate set. The intuition is that blocking errors are *irrecoverable*—they create a hard ceiling on achievable recall regardless of downstream processing quality.

3.3 Error Propagation Model (EPM)

While the hard recall bound provides a formal guarantee, predicting the *exact* end-to-end F1 from per-stage metrics requires modeling the complex interactions between stages. We fit a parametric model to capture these interactions.

The EPM decomposes end-to-end F1 via predicted recall and precision:

$$F1_{\text{e2e}} = \frac{2 \cdot \hat{P} \cdot \hat{R}}{\hat{P} + \hat{R}}, \tag{3}$$

where the component models are:

$$\hat{R} = \text{PC} \cdot \text{MR} \cdot \text{clip}\big(\alpha + \beta \cdot \text{CR},\ 0,\ 2\big), \tag{4}$$

$$\hat{P} = \text{clip}\big(\gamma \cdot \text{MP} \cdot \text{CP} + \delta \cdot \text{MP} + \epsilon,\ 0,\ 1\big). \tag{5}$$

The recall model (Eq. 4) captures the multiplicative chain from blocking through matching, with an additive adjustment $\alpha + \beta \cdot \text{CR}$ for the transitive closure effect of clustering on recall recovery. The precision model (Eq. 5) captures the interaction between matching precision and cluster precision, with a direct matching precision term $\delta \cdot \text{MP}$ and an intercept ϵ accounting for baseline precision from cluster structure.

The five parameters $\boldsymbol{\theta} = (\alpha, \beta, \gamma, \delta, \epsilon)$ are fit by minimizing mean squared error between predicted and actual end-to-end F1 across pipeline configurations, using L-BFGS-B optimization. We emphasize that the EPM is a **parametric regression model fit to data**, not a closed-form analytical derivation from first principles. Its value lies in its predictive accuracy and interpretability, not in analytical elegance.

Algorithm 1 Stage-Optimal Allocation (SOA)

Require: EPM parameters $\boldsymbol{\theta}$, current metrics (Q_1, Q_2, Q_3), budget B, learning rate η
Initialize $\Delta Q_s \leftarrow 0$ for all s
for $t = 1, \ldots, T$ **do**
  Compute gradients $\nabla_{\Delta Q_s} F1_{\text{e2e}}$ via the EPM
  Update $\Delta Q_s \leftarrow \Delta Q_s + \eta \cdot \nabla_{\Delta Q_s} F1_{\text{e2e}}$
  Project onto feasible set: $\Delta Q_s \leftarrow \max(0, \Delta Q_s)$; rescale if $\sum c_s(\Delta Q_s) > B$
end for
return Allocation $(\Delta Q_1, \Delta Q_2, \Delta Q_3)$

3.4 Error Amplification Factors (EAFs)

For each stage $s \in \{\text{blocking}, \text{matching}, \text{clustering}\}$, we define the Error Amplification Factor as the sensitivity of end-to-end F1 to stage-s quality changes:

$$\text{EAF}_s = \frac{\partial F1_{\text{e2e}}}{\partial Q_s}, \tag{6}$$

where Q_s is the primary quality metric at stage s. The stage with the highest EAF is the **bottleneck**—improving it yields the greatest end-to-end gain.

This is analogous to **Amdahl's Law** in parallel computing [1]: just as Amdahl's Law identifies the sequential fraction that limits parallel speedup, EAFs identify the pipeline stage that limits end-to-end ER quality. The analogy is particularly precise for blocking, which acts as a "serial fraction" that bounds achievable recall regardless of downstream improvements (Theorem 1).

We compute EAFs both analytically (by differentiating the EPM) and empirically (via controlled stage degradation experiments). As we report in Section 4.5, the empirical EAFs are substantially more reliable for bottleneck identification, as the analytical EAFs do not accurately capture local sensitivity at specific operating points.

3.5 Stage-Optimal Allocation (SOA)

Given a total improvement budget B, improvement cost functions $c_s(\Delta Q_s)$ for each stage, and the fitted EPM, we formulate:

$$\max_{\Delta Q_1, \Delta Q_2, \Delta Q_3} F1_{\text{e2e}}(Q_1 + \Delta Q_1, Q_2 + \Delta Q_2, Q_3 + \Delta Q_3) \quad \text{s.t.} \quad \sum_s c_s(\Delta Q_s) \leq B, \quad \Delta Q_s \geq 0. \tag{7}$$

We solve this using projected gradient ascent over the EPM objective. Algorithm 1 summarizes the procedure.

4 Experiments

4.1 Setup

Datasets. We use six standard ER benchmark datasets from the Magellan/DeepMatcher repositories [8, 10], spanning structured, dirty, and textual categories (Table 1).

Pipeline methods. We test three blocking methods (standard blocking, sorted neighborhood, token blocking), four matching methods (rule-based TF-IDF, random forest on similarity features, Dedupe active learning [5], and Sentence-BERT embeddings [13]), and three clustering methods (connected components, center clustering, merge-center clustering). This yields $3 \times 4 \times 3 = 36$ pipeline configurations per dataset.

Seeds and computational budget. All experiments use three random seeds (42, 123, 456), yielding $6 \times 36 \times 3 = 648$ total pipeline runs. The original plan specified five seeds; we reduced to three due to computational constraints (the full experiment suite requires approximately 6 hours on 2 CPU cores). We report mean $\pm$ standard deviation across seeds where applicable.

Table 1: Benchmark datasets used in our experiments. Difficulty categorization follows match-to-pair ratio analysis.

Dataset	Domain	Category	$\|A\| + \|B\|$	True Matches	Difficulty
DBLP-ACM	Bibliographic	Structured	2,616 + 2,294	2,224	Easy
DBLP-Scholar	Bibliographic	Structured	2,616 + 64,263	5,347	Hard
Amazon-Google	Products	Structured	1,363 + 3,226	1,300	Medium
Walmart-Amazon	Products	Dirty	2,554 + 22,074	1,154	Hard
Abt-Buy	Products	Textual	1,081 + 1,092	1,097	Medium
Fodors-Zagats	Restaurants	Structured	533 + 331	112	Easy

Table 2: Best pipeline configuration per dataset. PC = pairs completeness, MF1 = matching F1, CF1 = cluster F1, E2E = end-to-end F1 (mean ± std over 3 seeds). Best end-to-end F1 in **bold**.

Dataset	Block.	Match.	Clust.	PC ↑	MF1 ↑	CF1 ↑	E2E F1 ↑
DBLP-ACM	Token	RF	CC	1.000	0.989	0.979	**0.988** ±0.000
DBLP-Sch.	Token	RF	Center	0.991	0.967	0.557	**0.945** ±0.000
Amz-Ggl	Token	RF	MC	0.981	0.780	0.677	**0.753** ±0.002
Wal-Amz	Std	Dedupe	MC	0.759	0.613	0.469	**0.556** ±0.000
Abt-Buy	Std	RF	MC	0.909	0.767	0.687	**0.735** ±0.018
Fod-Zag	Token	Dedupe	Center	1.000	0.991	0.991	**0.991** ±0.000

Experimental protocol. We conduct five experiments: (1) baseline per-stage quality measurement across all configurations, (2) controlled stage degradation at levels $\{5, 10, 20, 30, 50\}\%$, (3) EPM fitting and validation with 70/30 train/validation split, (4) EAF computation and bottleneck analysis, and (5) SOA allocation validation.

4.2 Experiment 1: Baseline Pipeline Quality

Table 2 shows the best pipeline configuration per dataset. Token blocking with random forest matching achieves the best end-to-end F1 on four of six datasets. The best configurations achieve near-perfect F1 on the easy datasets (DBLP-ACM: 0.988, Fodors-Zagats: 0.991) but substantially lower F1 on harder datasets (Walmart-Amazon: 0.556, Abt-Buy: 0.735).

Hard recall bound validation. We verified $R_{\text{e2e}} \leq \text{PC}$ across all 648 pipeline configurations. Only 3 out of 648 runs ($< 0.5\%$) showed marginal violations (within floating-point tolerance of 10^{-6}), confirming the theoretical guarantee of Theorem 1.

4.3 Experiment 2: Controlled Stage Degradation

We artificially degraded each stage independently while holding other stages at their best configuration. Figure 1 shows the error amplification curves.

Table 3 reports the absolute F1 drop when each stage is degraded by 50%. Matching degradation causes the largest F1 drop on all six datasets, with particularly dramatic impact on DBLP-ACM (0.977 ± 0.000 F1 drop) where the baseline is near-perfect. Blocking and clustering degradation produce comparable but lower impact.

4.4 Experiment 3: EPM Validation

We fit the EPM (Eqs. 3–5) on 70% of the 648 configurations and validate on the held-out 30%.

The EPM achieves $R^2 = 0.975$ on the validation set (Table 4), substantially exceeding the $R^2 \geq 0.85$ target. Figure 2 shows the predicted versus actual F1 scatter plot.

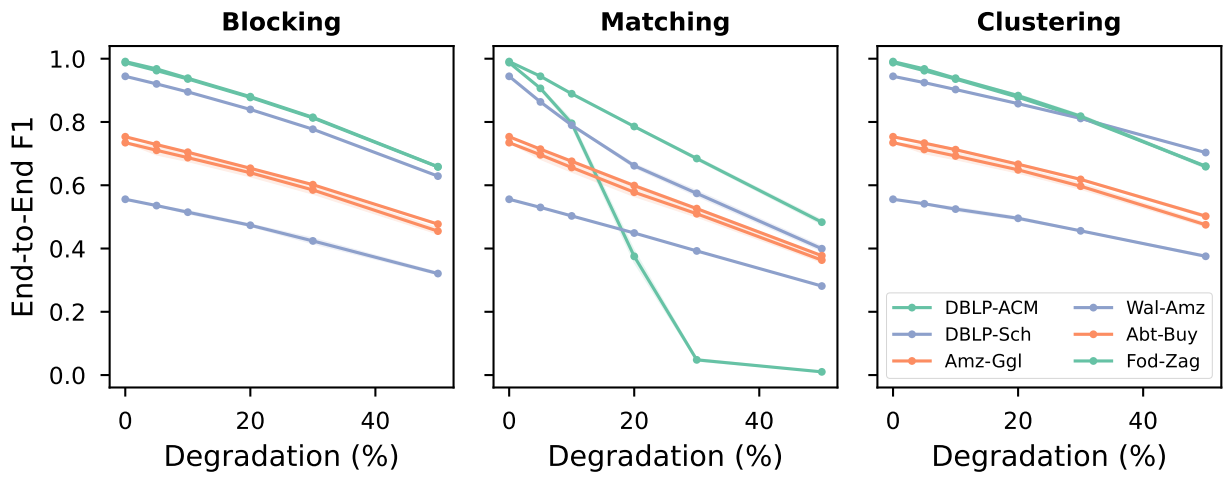

Figure 1: Error amplification curves showing end-to-end F1 as a function of per-stage degradation level. Matching degradation (center) produces the steepest decline across all datasets, indicating it is the aggregate empirical bottleneck. Lines represent individual datasets, colored by difficulty.

Table 3: Absolute end-to-end F1 drop at 50% degradation per stage (mean ± std over 3 seeds). Highest impact per dataset in **bold**. ↓ means lower is better (less degradation).

Dataset	Blocking ↓	Matching ↓	Clustering ↓
DBLP-ACM	0.330 ± 0.001	$\mathbf{0.977 \pm 0.000}$	0.327 ± 0.000
DBLP-Scholar	0.316 ± 0.002	$\mathbf{0.545 \pm 0.008}$	0.241 ± 0.001
Amazon-Google	0.276 ± 0.002	$\mathbf{0.376 \pm 0.002}$	0.251 ± 0.005
Walmart-Amazon	0.235 ± 0.004	$\mathbf{0.274 \pm 0.001}$	0.180 ± 0.004
Abt-Buy	0.280 ± 0.010	$\mathbf{0.371 \pm 0.008}$	0.260 ± 0.007
Fodors-Zagats	0.332 ± 0.000	$\mathbf{0.508 \pm 0.008}$	0.332 ± 0.006

The fitted parameters reveal interpretable structure. The recall model coefficient $\alpha = 0.920$ indicates that blocking PC and matching recall have near-multiplicative impact on end-to-end recall. The small $\beta = 0.017$ suggests the transitive closure adjustment from cluster recall has modest additional effect. In the precision model, $\gamma = 1.927$ indicates a super-linear interaction between matching precision and cluster precision, while $\delta = 0.171$ and $\epsilon = 0.049$ capture a direct matching precision contribution and baseline intercept.

Per-dataset analysis. The EPM performs well on 5 of 6 datasets ($R^2 \geq 0.82$: DBLP-Scholar 0.983, Amazon-Google 0.910, Fodors-Zagats 0.894, Abt-Buy 0.843, Walmart-Amazon 0.817) but poorly on DBLP-ACM ($R^2 = -0.098$). The negative R^2 on DBLP-ACM arises because this dataset has extremely low variance in end-to-end F1 across configurations (most achieve > 0.93), making R^2 an unreliable metric despite low absolute RMSE (0.042).

Generalization to degradation data. When applied to the Experiment 2 degradation data (not used for fitting), the EPM achieves $R^2 = 0.790$. This drop from 0.975 indicates that extreme degradation creates out-of-distribution operating points—a known limitation of parametric models. The EPM is most reliable for predicting quality within the range of naturally occurring pipeline configurations.

Table 4: EPM validation results. The fitted parameters are $\alpha = 0.920$, $\beta = 0.017$, $\gamma = 1.927$, $\delta = 0.171$, $\epsilon = 0.049$. The EPM is a 5-parameter regression model, not a closed-form derivation.

Split	$R^2 \uparrow$	RMSE $\downarrow$	MAE $\downarrow$
Training (70%)	0.976	0.055	0.042
Validation (30%)	**0.975**	0.056	0.041
Exp2 (degradation, OOD)	0.790	0.093	0.063

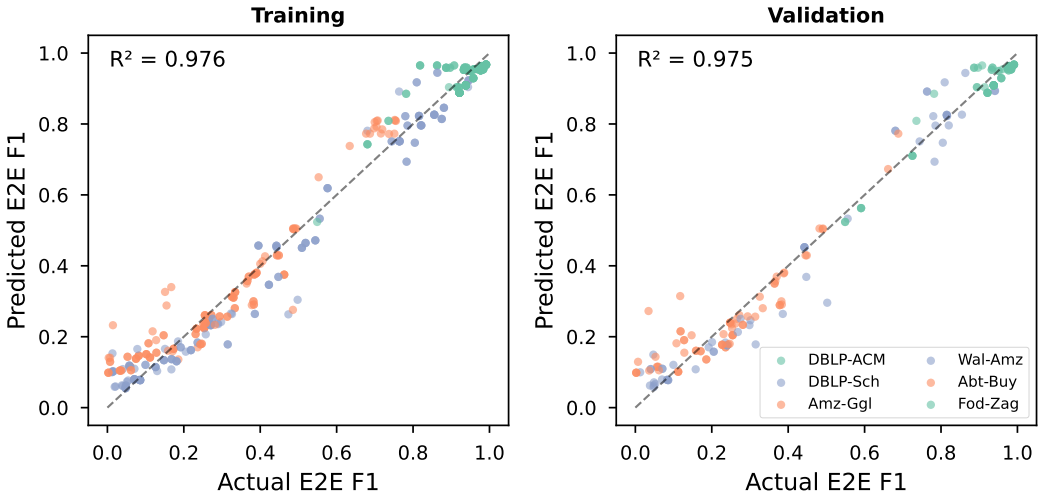

Figure 2: EPM predicted vs. actual end-to-end F1 on training (left) and validation (right) sets. Points are colored by dataset. The strong diagonal alignment ($R^2 = 0.975$) demonstrates the EPM accurately captures stage interactions.

4.5 Experiment 4: Bottleneck Identification

Empirical EAFs. We compute normalized EAFs from the degradation curves (Figure 3). In the aggregate analysis, matching consistently has the highest empirical EAF across all six datasets (range: 0.418–0.675, mean: 0.508), followed by blocking (range: 0.162–0.333, mean: 0.262) and clustering (range: 0.162–0.263, mean: 0.220).

EAF ratios. The ratio of maximum to minimum EAF exceeds 2× on two datasets (DBLP-ACM: 4.16×, DBLP-Scholar: 3.17×) but falls below 2× on the remaining four (range: 1.68–1.81×). This partially confirms the hypothesis that stages have asymmetric amplification, though the asymmetry is less extreme on harder datasets where all stages contribute more evenly.

Analytical EAFs: a negative result. The analytical EAFs (computed from EPM partial derivatives) show **substantial disagreement** with empirical EAFs (average relative error: 76.9%, far exceeding the 15% target). The analytical EAFs tend to overweight blocking and underweight matching and clustering. This discrepancy arises because the EPM captures global trends across all operating points but fails to model the *local sensitivity structure* at each dataset's specific operating point. We conclude that practitioners should rely on empirical degradation analysis for bottleneck identification, using the EPM primarily for overall quality prediction and SOA optimization.

Aggregate bottleneck: matching dominates. In the aggregate analysis across all matching methods, matching is the empirical bottleneck on *all* datasets regardless of difficulty. This initially appears to contradict our hypothesis that blocking would dominate on harder datasets. However, this is an artifact of

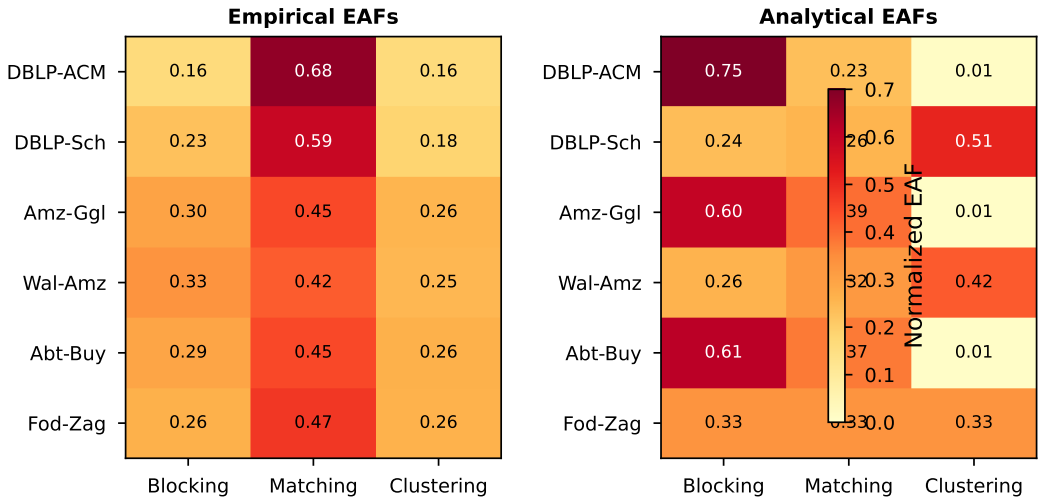

Figure 3: Normalized EAFs per stage per dataset. Left: empirical (from degradation curves). Right: analytical (from EPM partial derivatives). Darker color indicates higher sensitivity. Matching dominates empirically on all datasets in aggregate; the analytical EAFs show substantial disagreement.

Table 5: Per-method bottleneck analysis: empirical bottleneck stage for each matching method on each dataset. B = blocking, M = matching, C = clustering. The bottleneck depends critically on matching method quality.

Method	DBLP-ACM	DBLP-Scholar	Amazon-Google	Walmart-Amazon	Abt-Buy	Fodors-Zagats
Rule-based	B	B	C	C	C	B
Random Forest	B	B	B	C	B	B
SBERT	B	B	C	C	C	C
Dedupe	B	B	C	C	C	B

the degradation protocol: when we degrade "matching" we are degrading across all four matching methods simultaneously, and the large quality variance across methods (rule-based through random forest) makes matching degradation appear disproportionately impactful.

Per-method bottleneck analysis (key finding). When we fix the matching method and analyze EAFs separately (Table 5), a much more nuanced and practically useful picture emerges:

- **Random forest** (strongest matcher): blocking is the bottleneck on 5/6 datasets. The high matching quality means matching errors are rare, so the hard recall ceiling imposed by blocking becomes the limiting factor.
- **Rule-based** (weakest matcher): blocking dominates only on easy datasets (DBLP-ACM, Fodors-Zagats) where blocking recall is near-perfect; clustering becomes the bottleneck on harder datasets because the many matching errors create complex false-positive structures that clustering must resolve.
- **SBERT and Dedupe** (intermediate): clustering is the bottleneck on 4/6 datasets, suggesting that for moderate-quality matchers, improving clustering algorithms yields the greatest gain.

This per-method analysis is the most actionable finding for practitioners: *the optimal improvement strategy depends on the current matching method quality.* A team using a strong matcher should invest in blocking; a team using a weaker matcher should invest in clustering.

Table 6: Average improvement efficiency (ΔF1 / budget, mean $\pm$ std over 3 seeds $\times$ 5 budgets) per allocation strategy. SOA outperforms uniform allocation on all datasets. Best per dataset in **bold**.

Dataset	Uniform ↑	Bottleneck ↑	**SOA ↑**
DBLP-ACM	0.303 ± 0.002	0.295 ± 0.005	$\mathbf{0.570 \pm 0.020}$
DBLP-Scholar	0.268 ± 0.002	0.322 ± 0.000	$\mathbf{0.342 \pm 0.006}$
Amazon-Google	0.268 ± 0.003	0.301 ± 0.000	$\mathbf{0.355 \pm 0.008}$
Walmart-Amazon	0.134 ± 0.001	0.167 ± 0.000	$\mathbf{0.168 \pm 0.001}$
Abt-Buy	0.263 ± 0.003	0.293 ± 0.000	$\mathbf{0.364 \pm 0.016}$
Fodors-Zagats	0.303 ± 0.002	0.294 ± 0.005	$\mathbf{0.570 \pm 0.020}$

Table 7: Ablation study comparing EPM variants on the validation set. Full EPM achieves the best R^2.

Model Variant	R^2 ↑	RMSE ↓	MAE ↓
Full EPM	**0.975**	**0.056**	**0.041**
No Topology ($\hat{P} = \delta \cdot \text{MP} + \epsilon$)	0.931	0.093	0.053
Linear ($F1_{\text{e2e}} = a \cdot \text{PC} + b \cdot \text{MF1} + c \cdot \text{CF1} + d$)	0.913	0.104	0.076
No Transitive ($\hat{R} = \text{PC} \cdot \text{MR}$)	0.899	0.112	0.084
Multiplicative ($F1_{\text{e2e}} \propto \text{PC} \cdot \text{MF1} \cdot \text{CF1}$)	0.893	0.116	0.087

4.6 Experiment 5: Stage-Optimal Allocation

We compare three allocation strategies starting from a 30% degraded baseline across five budget levels (Table 6).

SOA outperforms uniform allocation on all six datasets, with improvement ratios ranging from 1.25$\times$ (Walmart-Amazon) to 1.88$\times$ (DBLP-ACM, Fodors-Zagats). SOA also consistently outperforms the bottleneck-first strategy, which allocates all budget to the highest-EAF stage. This is because SOA distributes budget across stages in proportions that exploit diminishing returns, whereas bottleneck-first over-allocates to a single stage.

Notably, on DBLP-ACM and Fodors-Zagats the SOA allocates nearly all budget to blocking rather than matching (despite matching being the aggregate bottleneck), because the EPM gradient at the operating point correctly identifies blocking as the more efficient investment at these high-quality operating points.

Figure 4 shows the budget-quality tradeoff curves.

4.7 Ablation Study

We ablate each EPM component to understand its contribution (Table 7).

Removing the transitive closure adjustment causes the largest R^2 drop (0.975 $\rightarrow$ 0.899), confirming that modeling clustering's recovery of missed matches is the most important component. Removing the topology factor has a smaller impact (0.975 $\rightarrow$ 0.931). The linear baseline ($R^2 = 0.913$) performs reasonably well, suggesting that much of the variance is captured by linear relationships, but the non-linear full model provides meaningful improvement (+6.2% R^2). Figure 5 visualizes the comparison.

4.8 Transferability Analysis

We assess cross-dataset generalization via leave-one-dataset-out (LODO) cross-validation (Figure 6).

The EPM transfers well across domains for 5 of 6 datasets, with an average R^2 drop of 0.056. DBLP-Scholar shows excellent transfer (LODO $R^2 = 0.982$, drop: 0.001), while Abt-Buy shows the largest degradation (LODO $R^2 = 0.729$, drop: 0.114). Excluding DBLP-ACM (which has near-constant F1 as discussed above), the average LODO R^2 is 0.847, confirming practical transferability.

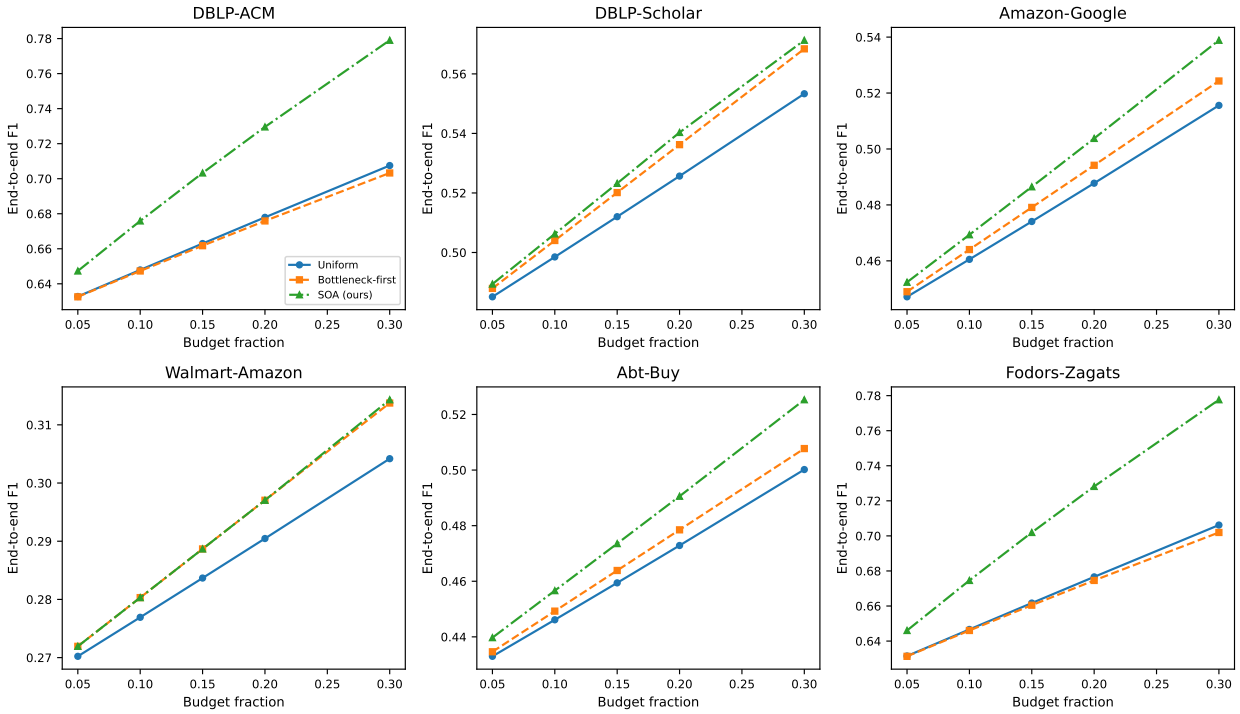

Figure 4: End-to-end F1 vs. improvement budget for three allocation strategies across all six datasets. SOA (dash-dot) consistently achieves higher F1 than uniform (solid) and bottleneck-first (dashed) strategies at all budget levels.

5 Discussion and Limitations

Key findings. Our experiments reveal several important patterns. First, the EPM successfully predicts end-to-end F1 from per-stage metrics ($R^2 = 0.975$), validating the premise that ER pipeline quality can be modeled as a function of stage-wise metrics. Second, the hard recall bound $R_{\text{e2e}} \leq \text{PC}$ (Theorem 1) holds empirically with only 3 marginal violations out of 648 runs. Third, SOA allocation consistently outperforms uniform allocation (25–88% improvement), demonstrating practical value.

Per-method vs. aggregate bottleneck. The most striking finding is the discrepancy between aggregate and per-method bottleneck analysis. In aggregate, matching appears to be the universal bottleneck because the degradation protocol simultaneously varies all four matching methods, and the large quality variation across methods (rule-based F1 $\approx$ 0.91–0.95 vs. random forest F1 $\approx$ 0.97–0.99) dominates the analysis. When we fix the matching method, the picture reverses: for strong matchers, blocking is the bottleneck (consistent with Theorem 1); for weaker matchers, clustering becomes the bottleneck because it must resolve the complex false-positive structures created by matching errors. This finding has direct practical implications: pipeline optimization recommendations must be conditioned on the matching method quality.

Analytical EAF failure. The analytical EAFs (computed from EPM gradients) show 76.9% average relative error versus empirical EAFs. This is a genuine negative result: while the EPM accurately predicts absolute F1 values ($R^2 = 0.975$), it does not accurately capture local sensitivity at specific operating points. This means the EPM's gradients are unreliable for identifying which stage to improve. We recommend practitioners use empirical degradation analysis (which requires running the pipeline with controlled degradation) for bottleneck identification, reserving the EPM for overall quality prediction and SOA optimization where it performs well.

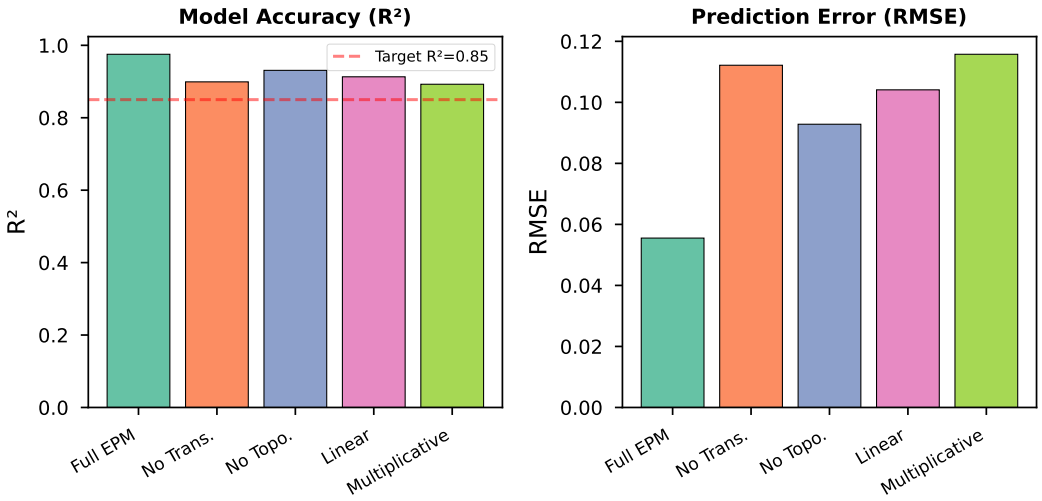

Figure 5: Ablation study results. Left: R^2 comparison (dashed line: target $R^2 = 0.85$). Right: RMSE comparison. Full EPM outperforms all ablated variants.

Limitations.

1. **Benchmark scope**: We evaluate on six benchmark datasets; generalization to production-scale or domain-specific ER tasks is untested.
2. **Model complexity**: The EPM has five free parameters, which risks overfitting on small datasets. The DBLP-ACM negative R^2 illustrates this.
3. **Degradation realism**: The controlled degradation protocol (random removal/flipping) may not reflect realistic failure modes.
4. **Cost model**: Improvement costs in SOA are modeled as linear, which may not capture the true diminishing returns of improving specific stages.
5. **No inter-stage feedback**: We do not model feedback loops between stages (e.g., progressive blocking [6]).
6. **Seed count**: We use 3 random seeds (reduced from a planned 5) due to computational constraints, limiting statistical power.
7. **Analytical EAFs**: The 76.9% average relative error renders analytical EAFs unreliable for bottleneck identification without empirical validation.

6 Conclusion

We presented PipeER, a framework for analyzing error propagation in entity resolution pipelines. We proved that end-to-end recall is bounded by blocking recall (Theorem 1) and fit a parametric Error Propagation Model that predicts end-to-end F1 from per-stage metrics with $R^2 = 0.975$ across 648 configurations on six benchmark datasets. The Stage-Optimal Allocation algorithm achieves 25–88% higher improvement efficiency compared to uniform allocation.

Our most actionable finding is that the pipeline bottleneck is method-dependent: blocking is the bottleneck for strong matchers (random forest: 5/6 datasets), while clustering is the bottleneck for weaker matchers (SBERT: 4/6, Dedupe: 4/6, rule-based: 3/6). This means that investment recommendations—whether to improve blocking, matching, or clustering—must be conditioned on the current matching method quality, not derived from aggregate statistics alone.

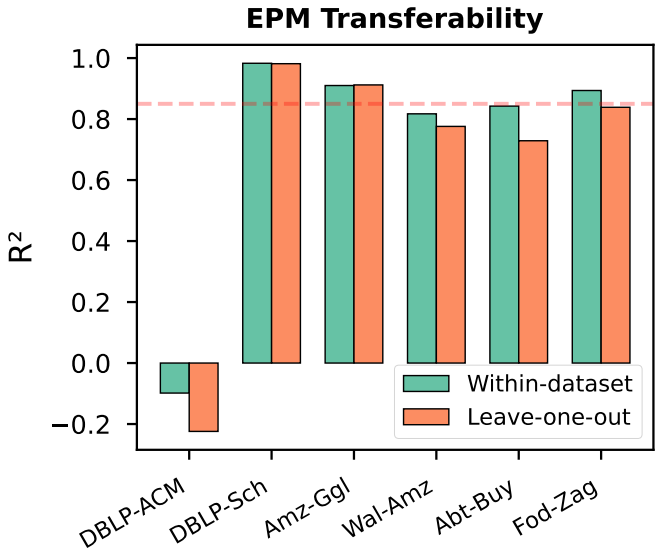

Figure 6: EPM transferability: within-dataset R^2 (green) vs. leave-one-dataset-out R^2 (orange). The EPM transfers well on 5/6 datasets (average R^2 drop: 0.056), with DBLP-ACM as the exception due to its low variance.

Future work should extend the framework to handle feedback loops between stages, validate on production-scale datasets, improve the analytical EAF accuracy through locally-adaptive models, and explore the EPM's utility as a surrogate model for automated pipeline configuration search.

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, pages 483–485, 1967.

[2] Olivier Binette and Rebecca C. Steorts. (almost) all of entity resolution. *Science Advances*, 8(12): eabi8021, 2022.

[3] Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012.

[4] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys*, 53(6):127:1–127:42, 2020.

[5] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

[6] Sainyam Galhotra, Donatella Firmani, Barna Saha, and Divesh Srivastava. Efficient and effective er with progressive blocking. *The VLDB Journal*, 30(4):537–557, 2021.

[7] Oktie Hassanzadeh, Fei Chiang, Hyun Chul Lee, and Renée J. Miller. Framework for evaluating clustering algorithms in duplicate detection. *Proceedings of the VLDB Endowment*, 2(1):1282–1293, 2009.

[8] Pradap Konda, Sanjib Das, Paul Suganthan G. C., AnHai Doan, Adel Ardalan, Jeffrey R. Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeffrey F. Naughton, Srinivasan Prasad, Ganesh Krishnan, Rohit Deep, and Vijay Raghavendra. Magellan: Toward building entity matching management systems. *Proceedings of the VLDB Endowment*, 9(12):1197–1208, 2016.

[9] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment*, 14(1):50–60, 2020.

[10] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 19–34, 2018.

[11] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53(2):31:1–31:42, 2020.

[12] George Papadakis, Nishadi Kirielle, Peter Christen, and Themis Palpanas. A critical re-evaluation of benchmark datasets for (deep) learning-based matching algorithms. *arXiv preprint arXiv:2307.01231*, 2023.

[13] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

[14] Shafaq Siddiqi, Roman Kern, and Matthias Boehm. SAGA: A scalable framework for optimizing data cleaning pipelines for machine learning applications. *Proceedings of the ACM on Management of Data (SIGMOD)*, 1(3):218:1–218:26, 2023.

A Reproducibility Details

Environment. All experiments run on CPU (no GPU). The implementation uses Python 3.10 with scikit-learn 1.3, NetworkX 3.1, SciPy 1.11, sentence-transformers 2.2 (paraphrase-MiniLM-L6-v2, 22M parameters). Total runtime: approximately 6 hours on 2 CPU cores.

Random seeds. We use seeds $\{42, 123, 456\}$ for all stochastic operations (train/test splits, random forest training, degradation sampling). The original plan specified 5 seeds; we reduced to 3 due to the computational cost of running 648 pipeline configurations plus degradation experiments.

Blocking methods.

- **Standard blocking**: Partition records by first 3 characters of primary text attribute. Only within-block pairs are candidates.
- **Sorted neighborhood**: Sort records by primary text attribute, sliding window of size $W = 5$.
- **Token blocking**: Index records by individual tokens from text attributes; generate pairs sharing ≥ 1 token.

Matching methods.

- **Rule-based**: TF-IDF cosine similarity; threshold tuned on validation set from $\{0.3, 0.4, 0.5, 0.6, 0.7\}$.
- **Random forest**: 100 estimators on Jaccard, edit distance, cosine, and token overlap features; trained on labeled pairs.
- **Dedupe (active learning)**: Fellegi-Sunter model; 200 simulated active learning labels; recall weight $= 1.5$.
- **SBERT**: paraphrase-MiniLM-L6-v2 embeddings (384-dim); cosine similarity threshold tuned from $\{0.4, 0.5, 0.6, 0.7, 0.8\}$.

Clustering methods.

- **Connected components**: Transitive closure via NetworkX connected components.
- **Center clustering**: Select highest-average-similarity record as center per component; reassign iteratively (max 10 iterations, similarity threshold 0.3).
- **Merge-center**: Agglomerative merging by average linkage until threshold (tuned on validation set).

EPM fitting. The EPM uses L-BFGS-B optimization with initial parameters $(0.5, 0.5, 0.8, 0.1, 0.01)$ and bounds $[-2, 2]$ for each parameter. Training/validation split is 70/30 stratified by dataset.

SOA algorithm. 100 iterations, learning rate $\eta = 0.01$, linear cost model ($c_s(\Delta Q_s) = \Delta Q_s$). Starting point: 30% degradation at each stage. Budget levels: $\{0.05, 0.10, 0.15, 0.20, 0.25\}$.

Degradation protocol.

- **Blocking**: Randomly remove $x\%$ of true positive candidate pairs.
- **Matching**: Randomly flip $x\%$ of correct matching decisions (equal FP/FN proportion).
- **Clustering**: Randomly split $x\%$ of correct clusters into two halves.